Socket Programming in C – Cheat Sheet

---

1. Header Files

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

#include <sys/types.h>
#include <sys/socket.h>

#include <netinet/in.h>
#include <arpa/inet.h>

#include <netdb.h>
```

---

2. Socket Functions

Function	Purpose
socket()	Create a socket
bind()	Assign IP address and port
listen()	Wait for client connections
accept()	Accept a client
connect()	Connect to server
send()	Send data
recv()	Receive data
write()	Write data
read()	Read data
sendto()	Send UDP packet

Function	Purpose
recvfrom()	Receive UDP packet
close()	Close socket

---

3. Create Socket

int sockfd;

sockfd = socket(AF_INET, SOCK_STREAM, 0);

- AF_INET → IPv4
- SOCK_STREAM → TCP
- SOCK_DGRAM → UDP

Returns

- >0 Success
- -1 Error

---

4. sockaddr_in Structure

struct sockaddr_in addr;

addr.sin_family = AF_INET;
addr.sin_port = htons(8080);
addr.sin_addr.s_addr = INADDR_ANY;

memset(addr.sin_zero, 0, sizeof(addr.sin_zero));

---

5. Important Address Functions

Any Local Interface

INADDR_ANY

Convert IP String → Binary

inet_pton(AF_INET, "127.0.0.1", &addr.sin_addr);

Binary → String

inet_ntop(...)

---

6. Bind (Server)

```
bind(sockfd,
   (struct sockaddr *)&addr,    sizeof(addr));
```

Associates socket with an IP address and port.

---

7. Listen

```
listen(sockfd, 5);
```

5 = Maximum pending connection requests (backlog)

---

8. Accept

```
int clientfd;

clientfd = accept(sockfd,
            NULL,
            NULL);
```

Returns a new socket for communicating with the client.

---

9. Connect (Client)

```
connect(sockfd,
      (struct sockaddr *)&addr, sizeof(addr));
```

Initiates a connection to the server.

---

10. Send Data

Using write()

```
write(sockfd, msg, strlen(msg));
```

Using send()

send(sockfd, msg, strlen(msg), 0);

---

11. Receive Data

Using read()

read(sockfd, buffer, sizeof(buffer));

Using recv()

recv(sockfd, buffer, sizeof(buffer), 0);

---

12. UDP

Send

sendto(sockfd, msg, strlen(msg), 0, (struct sockaddr *)&addr, sizeof(addr));

Receive

recvfrom(sockfd, buffer, sizeof(buffer), 0, (struct sockaddr *)&addr, &len);

---

13. Close Socket

close(sockfd);

---

14. Byte Order Functions

Function	Purpose
htons()	Host → Network (16-bit)
htonl()	Host → Network (32-bit)
ntohs()	Network → Host (16-bit)
ntohl()	Network → Host (32-bit)

Example

addr.sin_port = htons(8080);

---

15. Useful Functions

Clear buffer

memset(buffer, 0, sizeof(buffer));

Copy string

strcpy()

Compare

strcmp()

Length

strlen()

---

16. TCP Server Flow

socket()
  |
bind()
  |
listen()
  |
accept()
  |
read()/recv()
  |
write()/send()
  |
close()

---

17. TCP Client Flow

socket()
  |
connect()
  |
write()/send()
  |
read()/recv()

close()

---

18. UDP Server Flow

socket()
bind()
recvfrom()
sendto()
close()

---

19. UDP Client Flow

socket()
sendto()
recvfrom()
close()

---

20. Common Socket Constants

Constant	Meaning
AF_INET	IPv4
AF_INET6	IPv6
SOCK_STREAM	TCP
SOCK_DGRAM	UDP
INADDR_ANY	Any local IP
INADDR_LOOPBACK	Loopback (127.0.0.1)

Constant	Meaning
NULL	No address

---

21. Common Error Check Pattern

```
if (socketfd < 0) {
   perror("Socket");
   exit(EXIT_FAILURE);
}
```

---

22. One-Line Summary

Function	Description
socket()	Create a communication endpoint
bind()	Attach socket to IP and port
listen()	Mark socket as passive (server)
accept()	Accept a client connection
connect()	Connect to a server
send() / write()	Send data
recv() / read()	Receive data
sendto()	Send UDP datagram
recvfrom()	Receive UDP datagram
close()	Release the socket
htons()	Convert 16-bit host to network byte order
htonl()	Convert 32-bit host to network byte order
ntohs()	Convert 16-bit network to host byte order
ntohl()	Convert 32-bit network to host byte order